Serverless applications are typically event-driven and run only when there is an event to handle, such as an HTTP request or a message. The event can be external or be produced by another function. For example, a function might be triggered whenever a message is added to a queue, process it, and then exit the execution.

When there is nothing to process, the serverless platform shuts down all the resources involved with the function, so you can really pay for the actual usage. In the other cloud native topologies like CaaS or PaaS, there is always a server running 24/7. Compared to traditional systems, they offer the advantage of dynamic scalability, reducing the number of resources provisioned at any given time. Still, there is always something up and running, and it has a cost. In the serverless model, resources are provisioned only when necessary. If there is nothing to process, everything is shut down. That's what we call *scaling to zero*, and it's one of the main features offered by serverless platforms.

Besides cost optimization, serverless technologies also move some extra responsibility from the application to the platform. That might be an advantage, since it allows developers to focus exclusively on the business logic. But it's also essential to consider what degree of control you would like to have and how you will deal with vendor lock-in. Each FaaS, and in general serverless, platform has its own features and APIs. Once you start writing functions for a specific platform, you can't move them easily to another, as you would do with containers. With FaaS, you might compromise more than with any other approach—prioritizing responsibility and scope at the cost of control and portability. That's why Knative became popular quickly: it is built on Kubernetes, which means that you can easily move your serverless workloads between platforms and vendors. In the end, it's a matter of tradeoffs.